

Teórica 5
UML

Why did the developer go broke?



Sumário

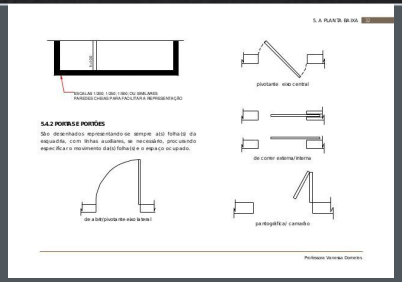
- Desenho de Sistemas de Software
- Modelos
- Introdução ao UML
- Diagramas de Classes e Objectos
  - Objectos
  - Classes
  - Relações



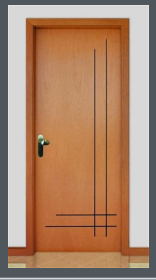
Desenho

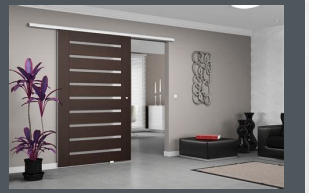
Requisitos: Mecanismo que nos permite passar por paredes.

Desenho:



Implementação:







Desenho

O processo de passar do domínio do problema para o domínio da solução.

Este processo produz uma descrição de uma solução do problema que satisfaz os requisitos.

Modelos

... e Engenharia Orientada a Modelos (uso de modelos na engenharia)

Modelos como cidadãos de primeira classe

O que é um modelo?



"Um modelo é uma abstracção de um sistema (real ou baseado em linguagem) que permite fazer previsões ou inferências."

Thomas Khune, Matters of (Meta-) Modeling, SoSYM, 2006

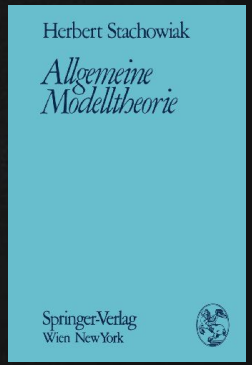
Por outras palavras

Os modelos suportam a representação de um sistema reduzida ao essencial de forma a minimizar a complexidade do sistema a aspectos mais fáceis de gerir.

Um sistema é normalmente descrito como um número de vistas que, em conjunto, produzem uma imagem geral unificada.

Herbert Stachowiak – Um modelo é distinguido por três características:

1. Mapeamento
2. Redução
3. Pragmatismo



Allgemeine Modelltheorie" (General Model Theory), Herbert Stachowiak, 1973

Herbert Stachowiak – Um modelo é distinguido por três características:

1. Mapeamento:

   Um modelo é sempre uma imagem (mapeamento) de algo, uma representação, natural ou artificial, de um original que pode ser, por sua vez, também um modelo.

2. Redução
3. Pragmatismo

Allgemeine Modelltheorie" (General Model Theory), Herbert Stachowiak, 1973

Herbert Stachowiak – Um modelo é distinguido por três características:

1. Mapeamento
2. Redução

   Um modelo não captura todos os atributos do original, mas apenas aqueles que parecem relevantes para quem modela ou para o utilizador do modelo.
3. Pragmatismo

Allgemeine Modelltheorie" (General Model Theory), Herbert Stachowiak, 1973

Herbert Stachowiak – A model is distinguished by three characteristics:

1. Mapeamento
2. Redução
3. Pragmatismo

   Pragmatismo significa orientação para utilidade. Um modelo é atribuído a um original com base nas seguintes questões: Para quem? Porquê? Para quê? Um modelo é utilizado por quem o elabora ou por um utilizador, em vez do original, dentro de um período de tempo específico e com um propósito específico.

Allgemeine Modelltheorie" (General Model Theory), Herbert Stachowiak, 1973

Bran Selic, características para determinar a qualidade de um modelo

- Abstracção
- Perceptibilidade
- Expressividade
- Precisão
- Relação custo-benefício

Forward e backward engineering

Modelos como rascunhos (informais) e planos (mais rigorosos) podem ser utilizados tanto em *forward engineering* como em *backward engineering*.

Forward engineering

O modelo é a base para criar o código.

Backward engineering

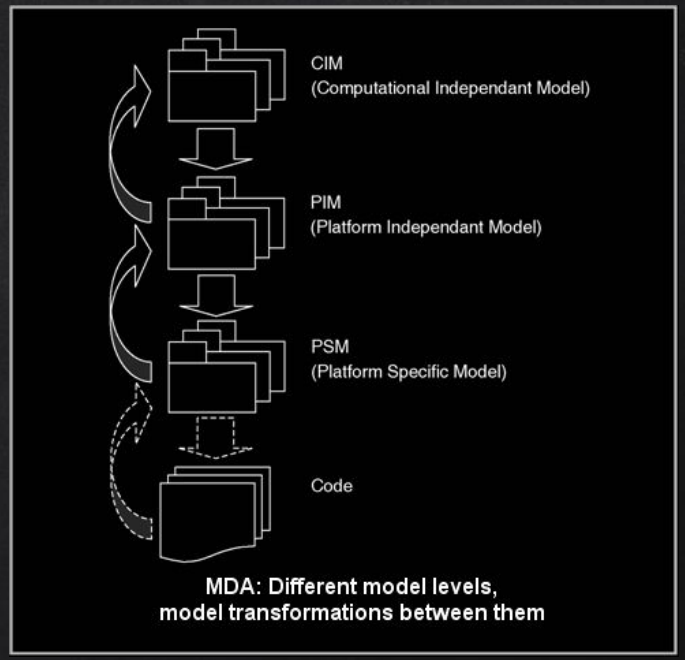
O modelo é gerado a partir do código para o documentar de uma forma clara e facilmente compreensível.

Models as first-class citizens

Modelos visuais
são formas excelentes para documentar, e mais...

Software orientado a Modelos – a visão da OMG para o desenvolvimento orientado a modelos

Promove o uso de modelos (e traduções automáticas) desde os requisitos até o código como forma de documentar e recolher informações essenciais para a perspectiva dos sistemas (num nível adequado de abstração)



MDA: Different model levels, model transformations between them

UML

Unified Modeling Language

Criadores do UML- os três amigos



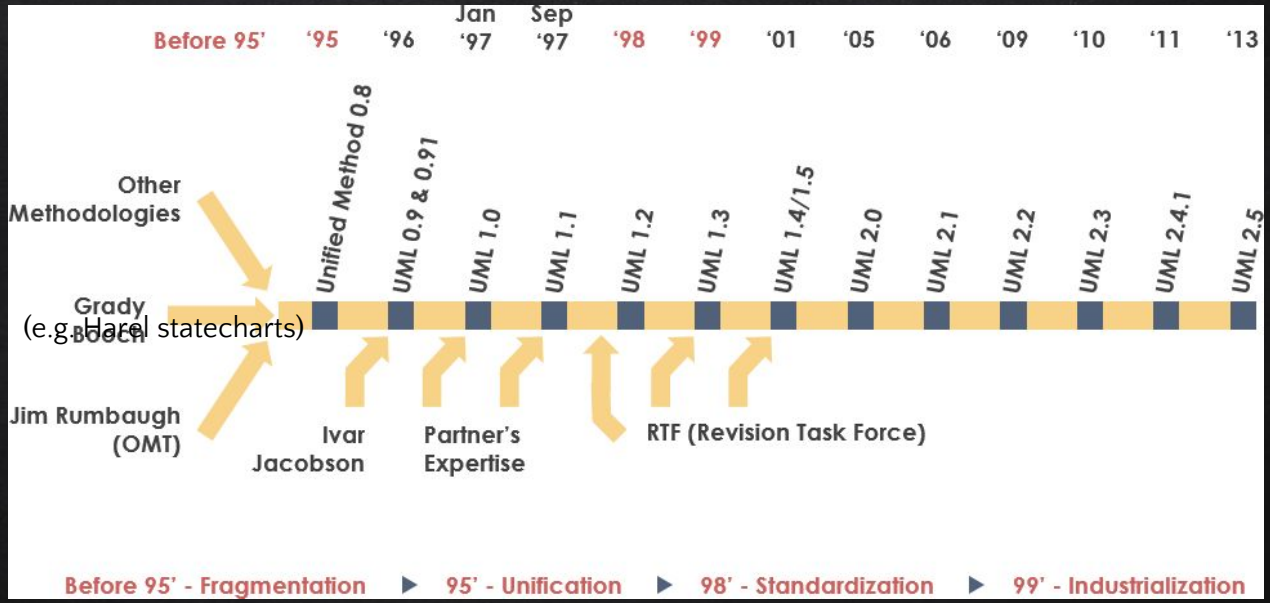
Criadores do UML- os três amigos



Ok, ok, não são estes três amigos... ☺️



História do UML



Fonte: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/

UML

- Linguagens de modelação visual de uso geral
- Podem ser utilizadas em diferentes processos de engenharia
- Apoia o ciclo de vida do desenvolvimento, desde os requisitos até à implementação
- Modelação de sistemas para vários domínios*
  - incluindo sistemas incorporados, em tempo real e de gestão e apoio à decisão
- Principalmente associada à modelação orientada a objectos, mas vai além disso e é independente de linguagem e plataforma.

Nota: UML NÃO é uma metodologia!

*SYSML é uma extensão do UML (via UML Profiles) para fins de Engenharia de Sistema

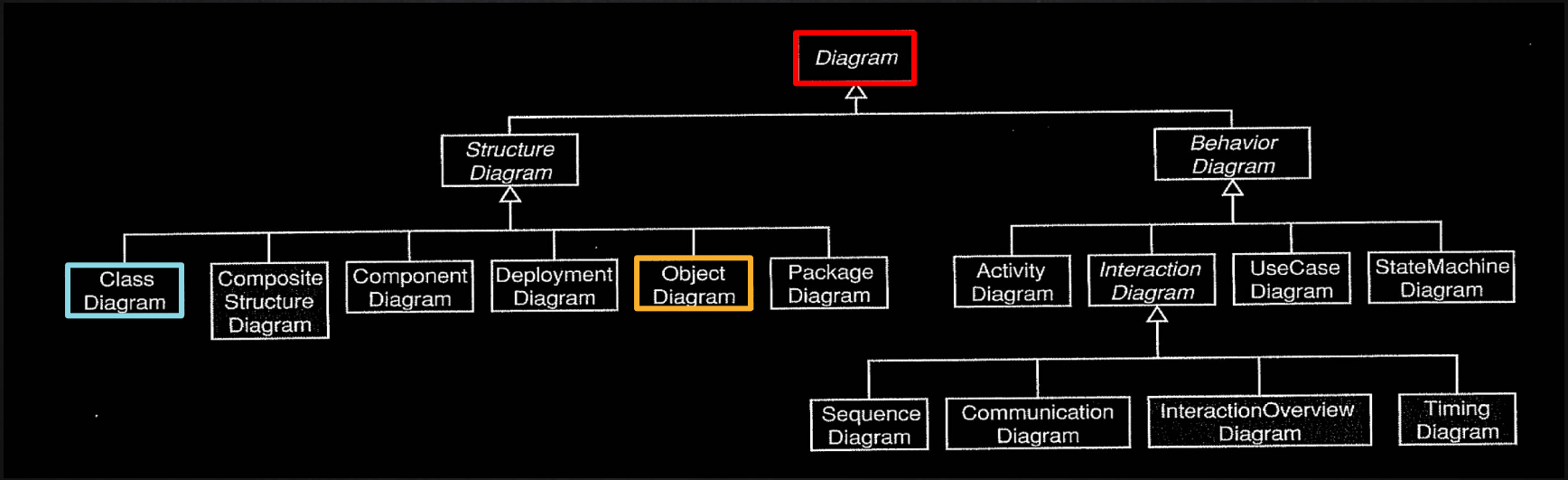
Objects e UML

- A UML modela sistemas intensivos em software como coleções de objectos
- São utilizadas duas estruturas coesas:
  - Estrutura estática – descreve os tipos de objectos e as suas relações
  - Estrutura dinâmica – descreve os ciclos de vida dos objectos e as interações entre esses objectos, de modo a satisfazer os requisitos do sistema

Elementos básicos do UML

- Coisas (elementos e nomes do modelo)
- Relações (como as coisas se relacionam)
- Diagramas (vistas do modelo)

Diagramas UML



Já conhecem as classes
Já trabalham com elas há 3 anos...

Tenham presente tudo o que sabem sobre elas. Isso continua a ser relevante.

Mantenham a mente aberta. Hoje irão vê-las sob uma perspectiva diferente.

Objectos

(lembrete rápido do que aprendeu em Introdução à Programação, só por precaução...)

O QUE É UM OBJECTO?

Uma entidade discreta com limites bem definidos que encapsula estado e comportamento; uma instância de uma classe.

- Os objectos combinam dados e funções numa unidade coesa
- Os objectos ocultam dados atrás de uma camada de operações (encapsulamento)

Encapsulamento

Proteção contra acesso não autorizado ao estado interno de um objecto por meio de uma interface única e bem definida.

Diferentes níveis de visibilidade das interfaces ajudam a definir diferentes autorizações de acesso. Tal como em Java:

Modificador de nível de acesso Java	Classe	Pacote	Subclasse	Mundo
public	✅	✅	✅	✅
protected	✅	✅	✅	❌
(default)	✅	✅	❌	❌
private	✅	❌	❌	❌

Propriedades do objecto

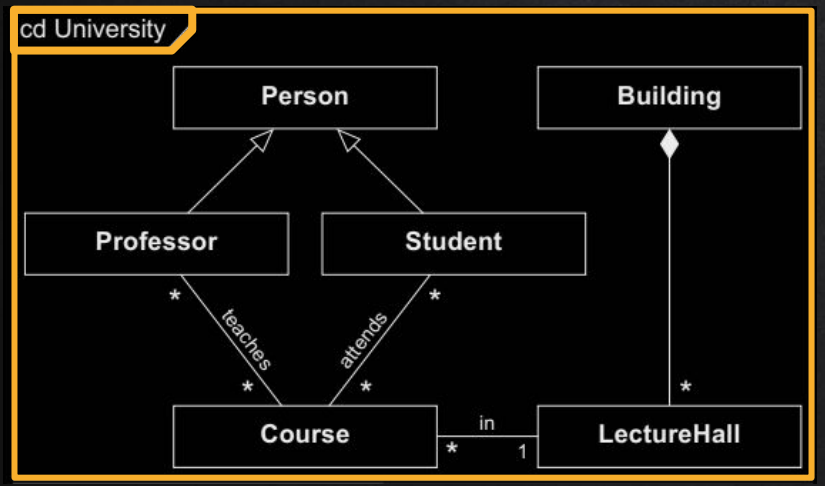
- Identidade
  - Cada objecto é identificável de forma única
- Estado
  - O estado atual de um objecto é determinado pelos valores dos seus atributos e pelas relações com outros objectos num determinado momento
- Comportamento
  - O comportamento de um objecto é caracterizado pelas suas operações
    - Algumas operações modificam o estado do objecto
    - Outras operações permitem consultar o objecto sobre o seu estado

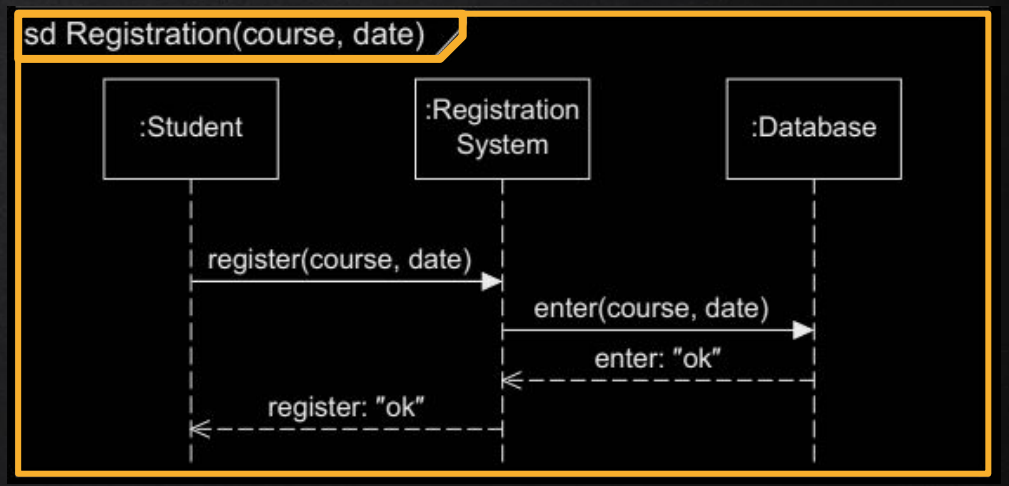
Os objectos colaboram para gerar o comportamento do sistema

- Os objectos formam ligações entre si e enviam mensagens através dessas ligações
- Quando um objecto recebe uma mensagem, verifica o seu conjunto de operações procurando uma operação cuja assinatura corresponda à da mensagem
  - Se existir uma correspondência, o objecto invoca a operação e, possivelmente, devolve um resultado para o objecto que enviou a mensagem

Notação para uma moldura de diagrama

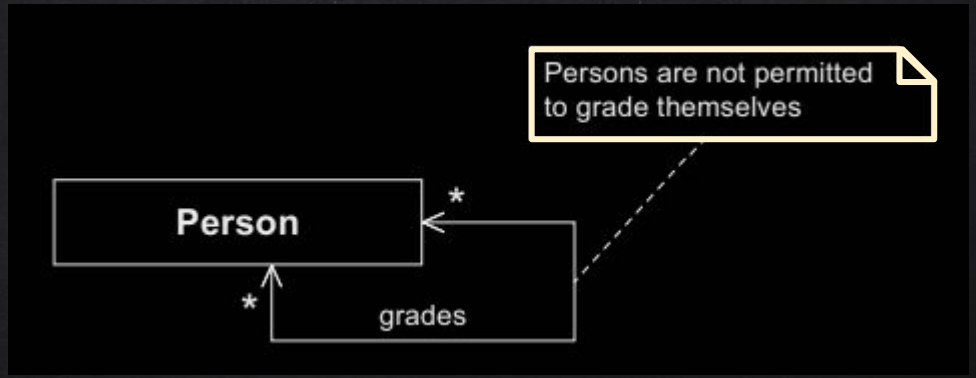
Um diagrama é geralmente delimitado por um retângulo com um pentágono no canto superior esquerdo. Este pentágono contém o tipo de diagrama e o nome da moldura (*frame*) do diagrama.



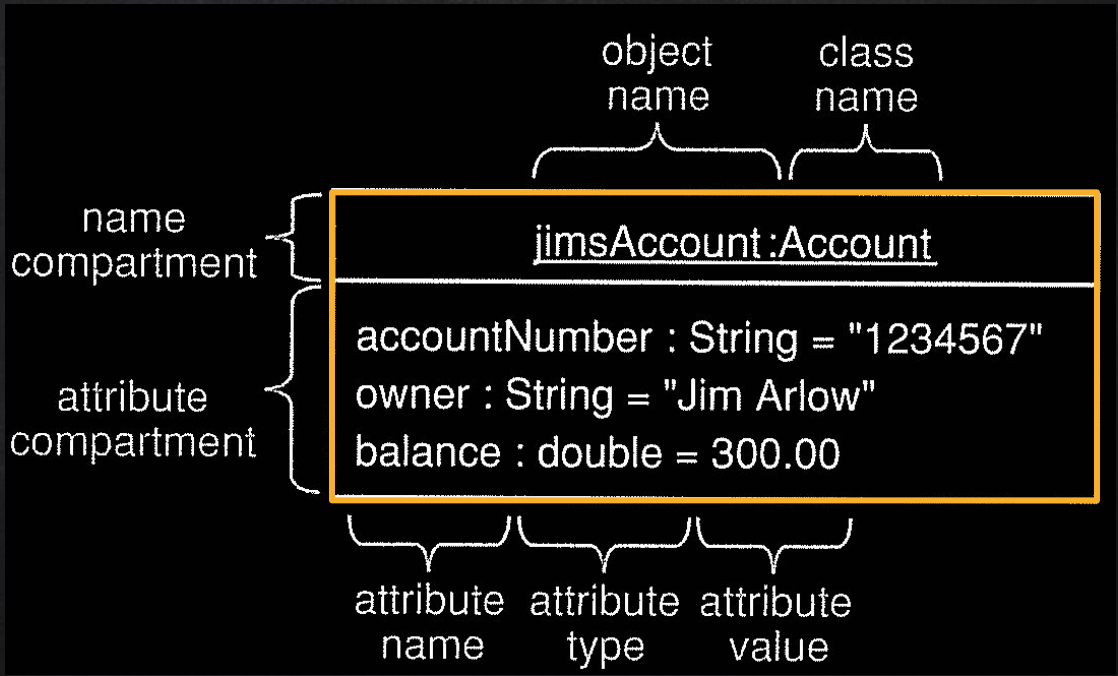


Nota

Uma nota pode conter qualquer forma de expressão que especifique o diagrama e os seus elementos com mais precisão. As notas podem estar ligadas a todos os outros elementos do modelo.



Objectos em UML



Notações alternativas para Objectos



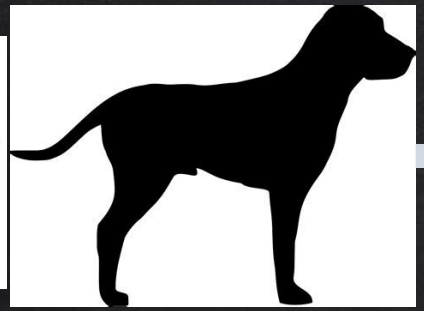
Classes

(lembrete rápido do que aprendeu em Introdução à Programação, só por precaução...)

O que é uma classe?

Descritor para um conjunto de objectos que partilham os mesmos atributos, operações, métodos, relações e comportamento

Properties	Methods
Color	Sit
Eye Color	Lay Down
Height	Shake
Length	Come
Weight	



Instância

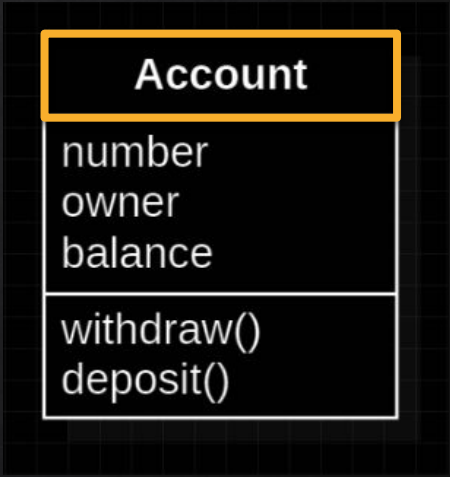


Property Values	Methods
Color: Yellow	Sit
Eye Color: Brown	Lay Down
Height: 17 in	Shake
Length: 35 in	Come
Weight: 24 pounds	

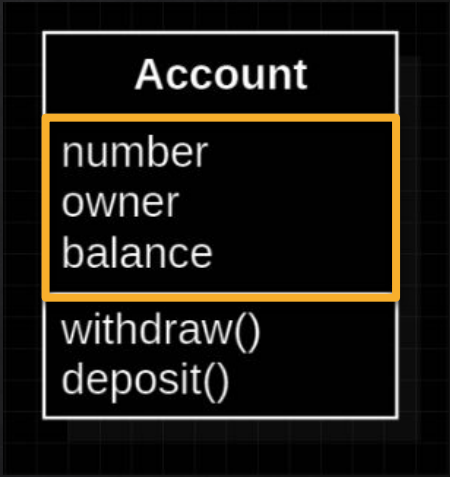
Cada objecto é uma instância de exatamente uma classe

- Os objectos da mesma classe partilham uma estrutura comum
  - O mesmo conjunto de operações
  - O mesmo conjunto de atributos (mas valores diferentes)
  - O mesmo conjunto de relações

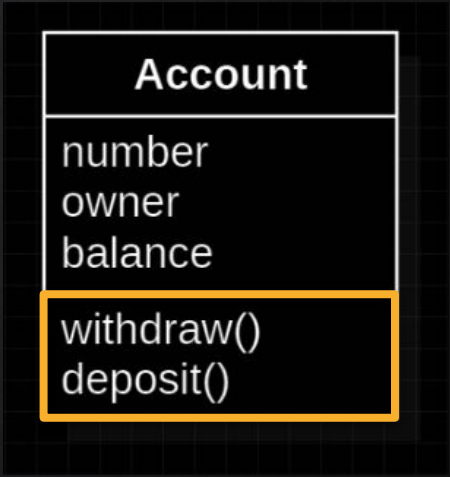
Uma classe tem um nome



Uma classe tem atributos



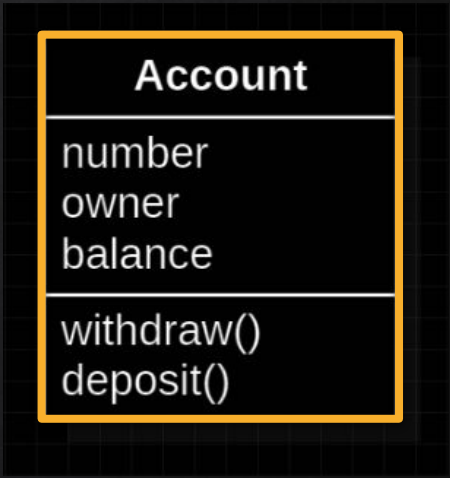
Uma classe tem operações



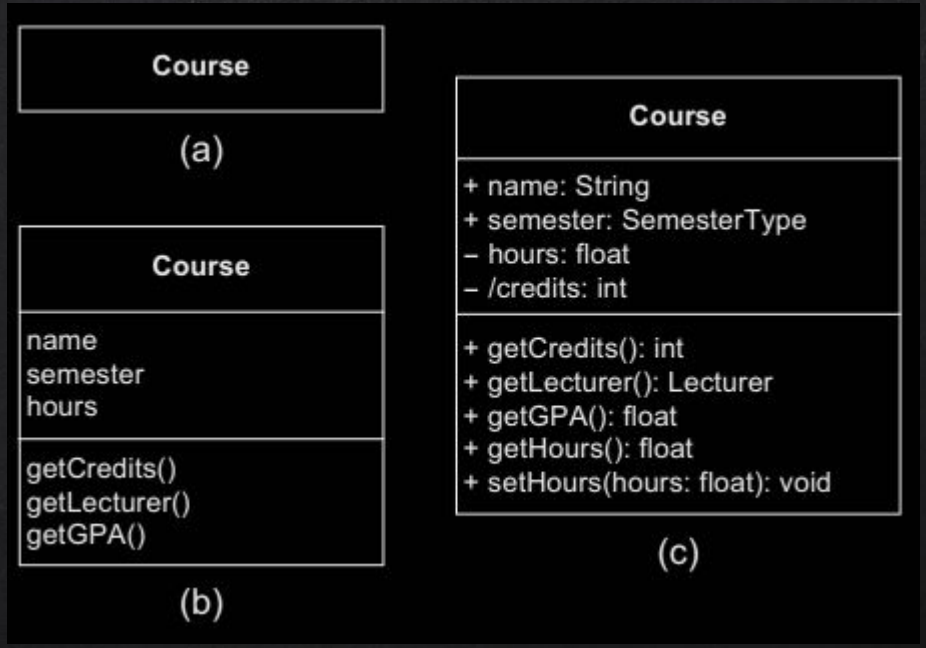
Uma classe pode ocultar muitos detalhes

Por enquanto, os seguintes elementos estão ocultos:

- Visibilidade de atributos e operações
- Estereótipos – mostrados apenas se ajudarem a compreender o modelo
- *Tagged values* – mostrados apenas se ajudarem a compreender o modelo

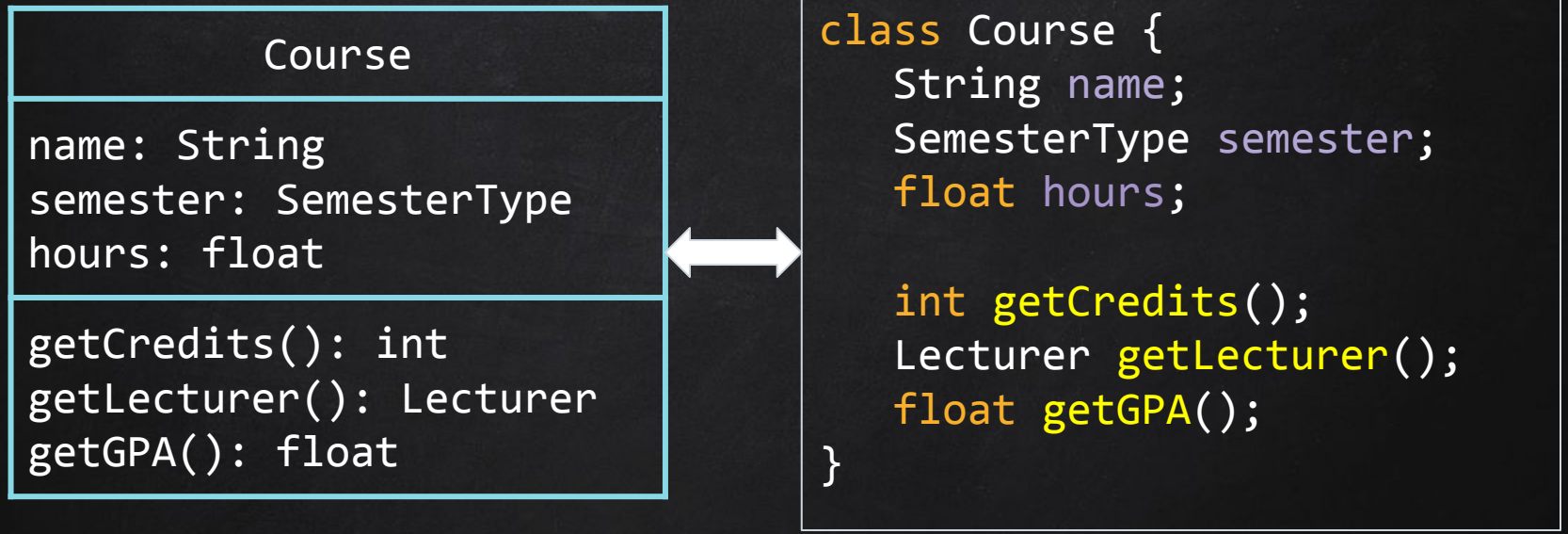


Uma classe pode ser representada com diferentes níveis de detalhe



Os atributos e operações são normalmente acedidos através dos seus nomes (identificadores). Esses identificadores seguem as convenções de nomenclatura *camelCase*, começando por uma letra minúscula.

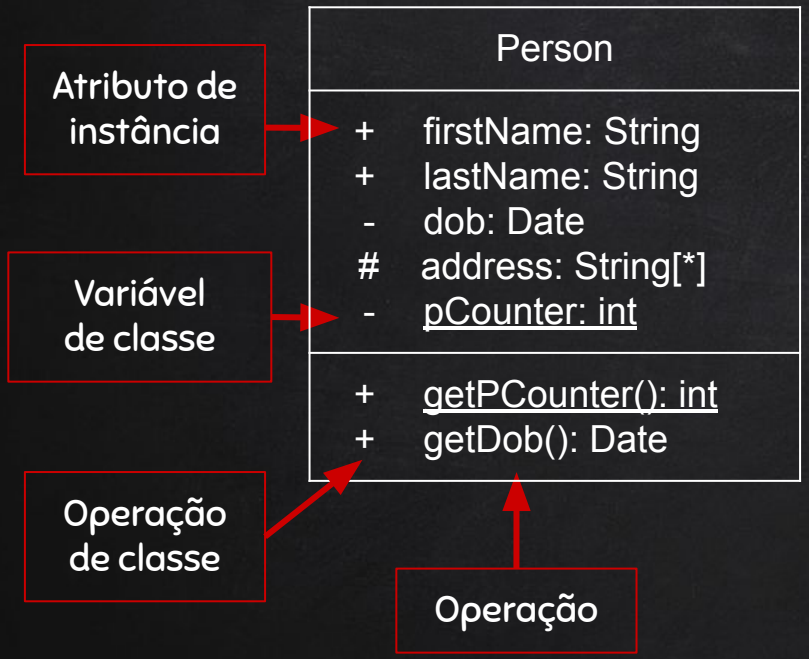
Definição de classe em UML e Java



Visibilidade de atributos e operações

Modificador de nível de acesso UML	Semântica	Classe	Pacote	Subclasse	Mundo
+	público	✅	✅	✅	✅
#	protegido	✅	✅	✅	❌
~	pacote	✅	✅	❌	❌
–	privado	✅	❌	❌	❌

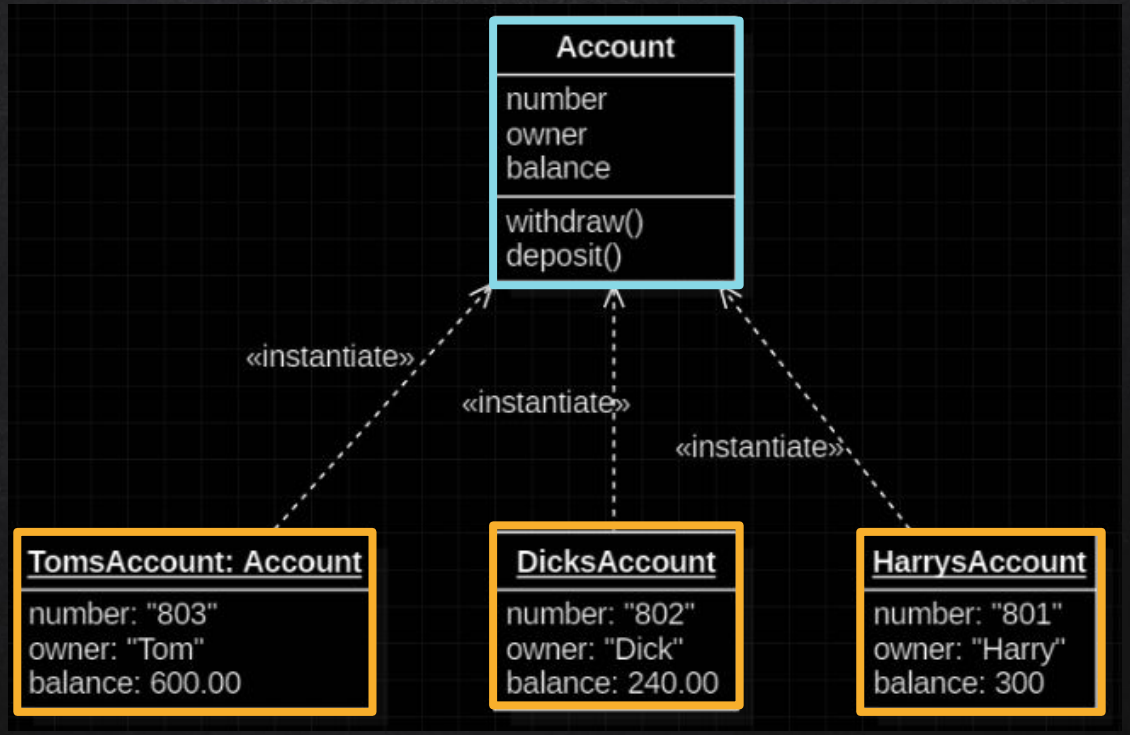
Tradução de uma classe de UML para Java




```java
class Person {
    public String firstName;
    public String lastName;
    private Date dob;
    protected String[] address;
    private static int pCounter;

    public static int getPCounter() {...}
    public Date getDob() {...}
}
```

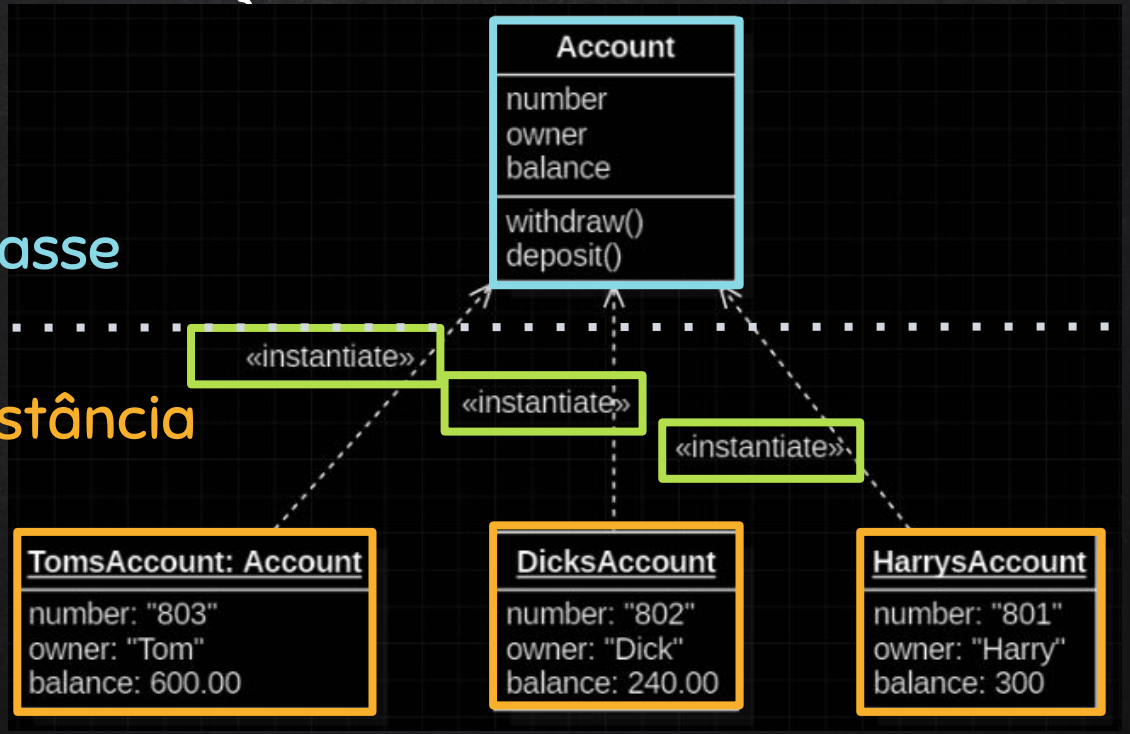
Os objectos são instanciações de classes



O estereótipo «instantiate» classifica o tipo de associação entre objectos e classes

Nível de Classe

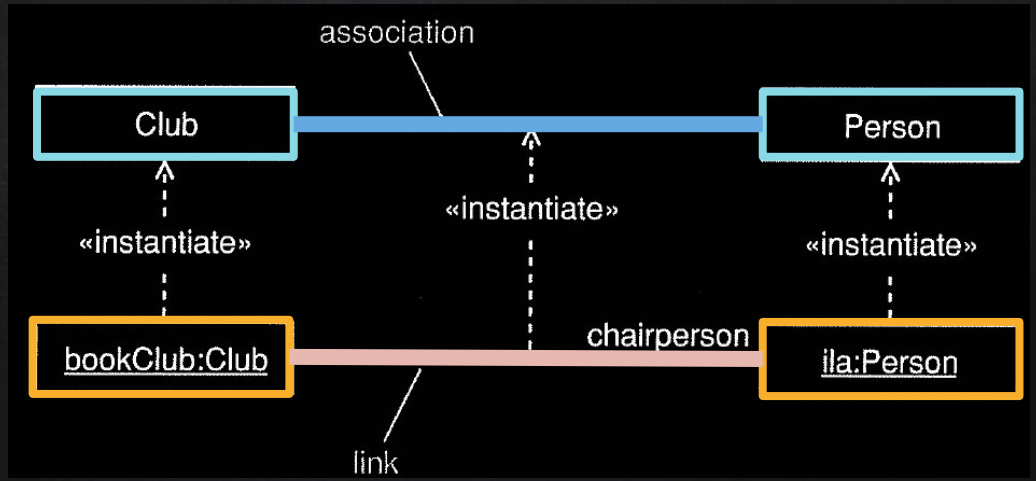
Nível de Instância



Relações

Relações entre classes e objectos

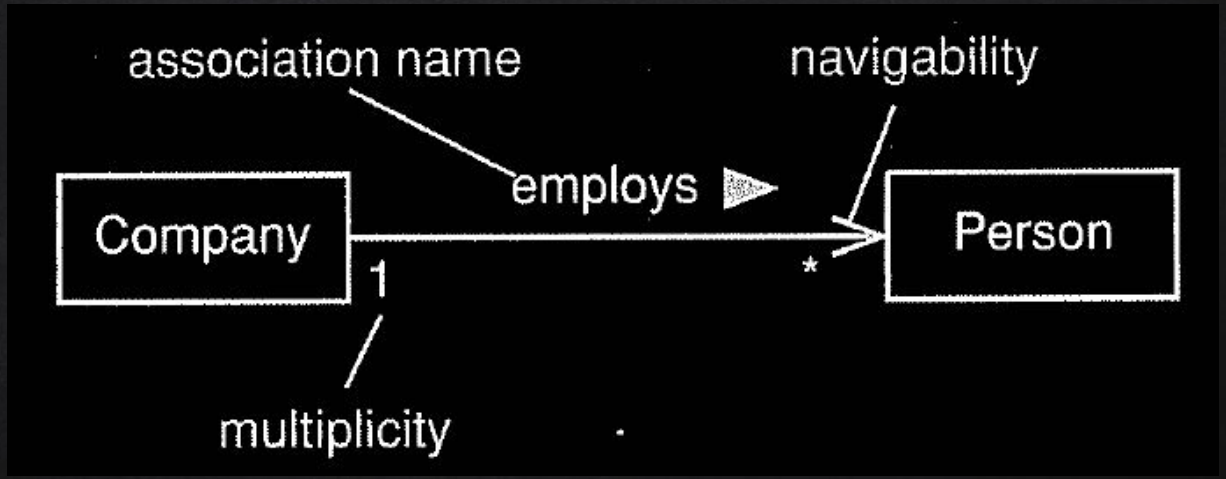
- Uma associação é uma relação entre classes
- Um link é uma relação entre objectos
- Um link é uma instanciação de uma associação
- Os objectos instanciam classes da mesma forma que os links instanciam associações



Sintaxe da associação

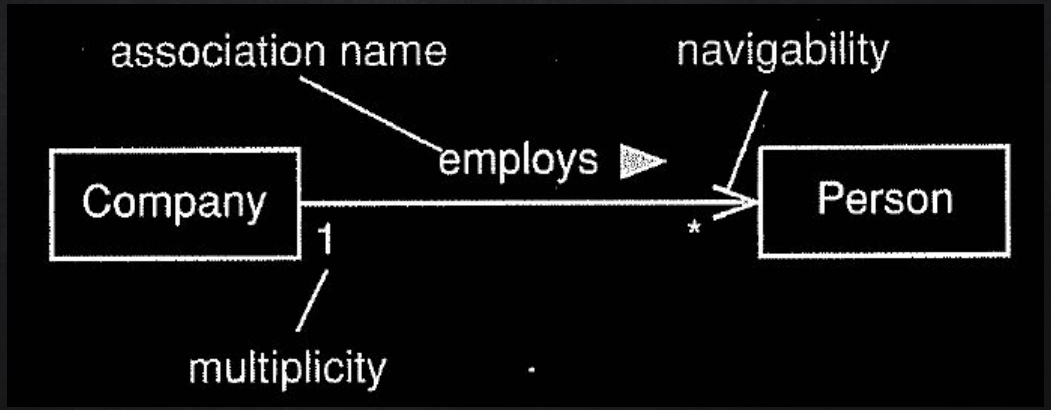
- Nome da associação (*association*)
  - Frase verbal, indicando uma ação do objecto de origem para o objecto de destino (ou seja, a semântica da associação)
    - Pode ser anotada com uma pequena seta preta para indicar a direção
- Nome do papel (*role*)
  - Frase nominal indicando a função dos objectos ligados por instâncias da associação
- Multiplicidade
  - Restringe o número de objectos de uma classe que participam numa relação em qualquer momento
- Navegabilidade
  - Se forem unidirecionais, as associações devem incluir uma seta

"Uma empresa emprega muitas pessoas"
"Uma pessoa é empregada por exatamente uma empresa"

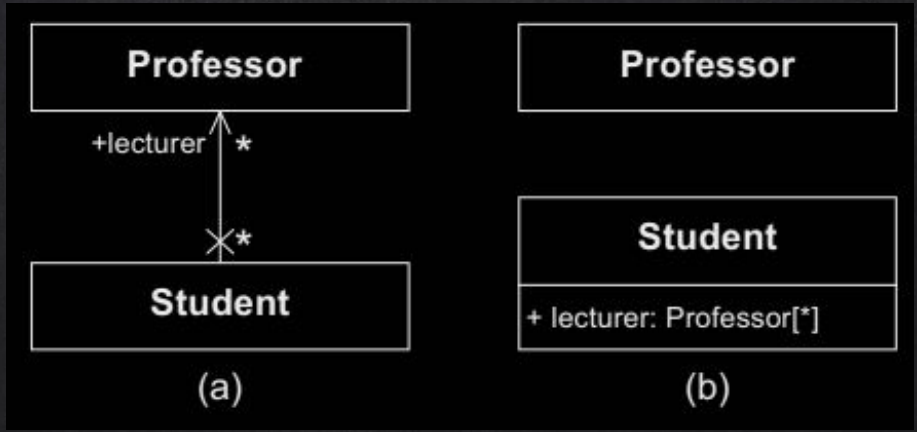


A multiplicidade restringe o número de objectos de uma classe que participam de uma relação em qualquer momento.

- Os objectos *Person* só podem ser empregados por uma empresa num determinado momento
- Os objectos *Person* devem ser sempre empregados por uma empresa
- Ao longo do tempo, um objecto *Person* pode estar empregado por diferentes empresas



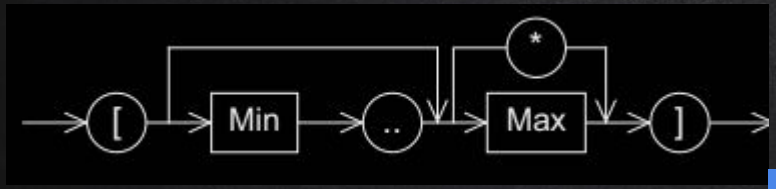
Associações em UML e Java




```
class Professor { ... }

class Student {
    public Professor[] lecturer;
    ...
}
```

Que tipos de multiplicidades podemos definir?



Estes adornos de multiplicidade devem ser colocados ao lado das classes correspondentes.

Ao ler um modelo, pode utilizá-los para expressar corretamente a intenção do modelo.

Adorno	Semântica
0..1	0 ou 1
0..*	0 ou mais
*	0 ou mais
1	Exatamente 1
1..*	1 ou mais
1..4	1 a 4
1..3, 7..10, 22	1 a 3, 7 a 10, 22

Certifique-se de que lê associações e multiplicidades com precisão



Uma empresa emprega exactamente 7 funcionários

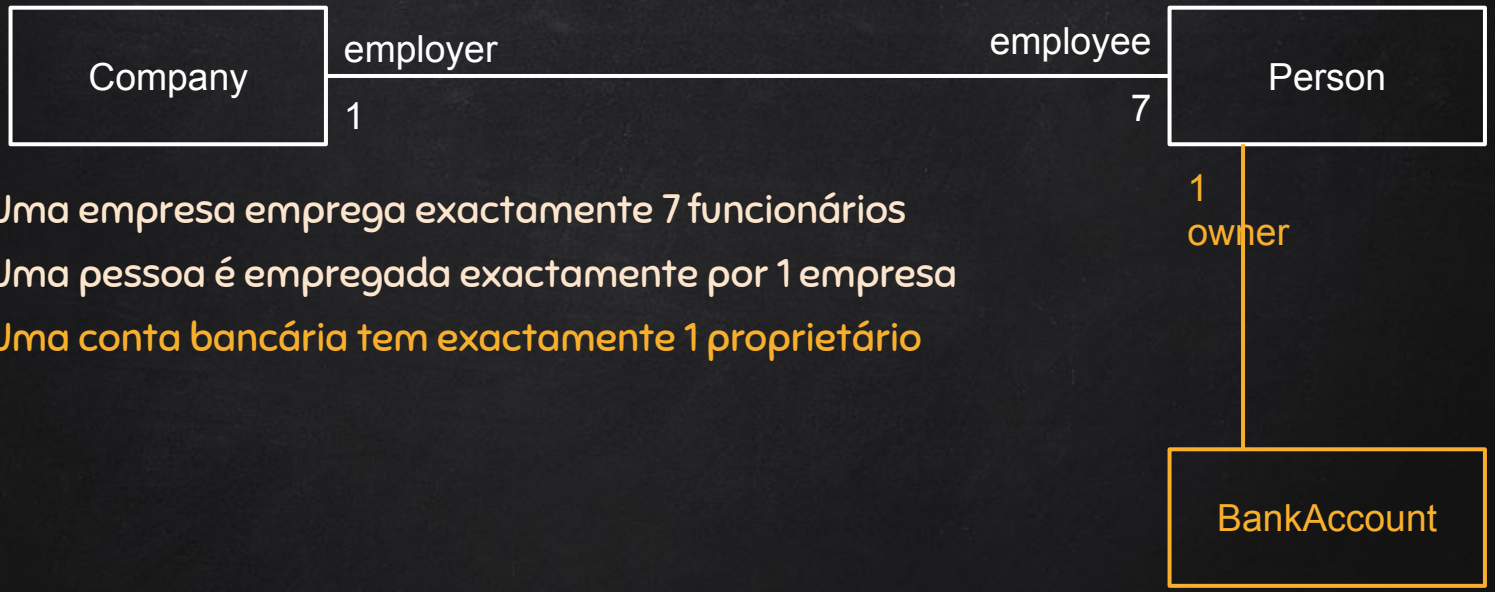
Certifique-se de que lê associações e multiplicidades com precisão



Uma empresa emprega exactamente 7 funcionários

Uma pessoa é empregada exactamente por 1 empresa

Certifique-se de que lê associações e multiplicidades com precisão

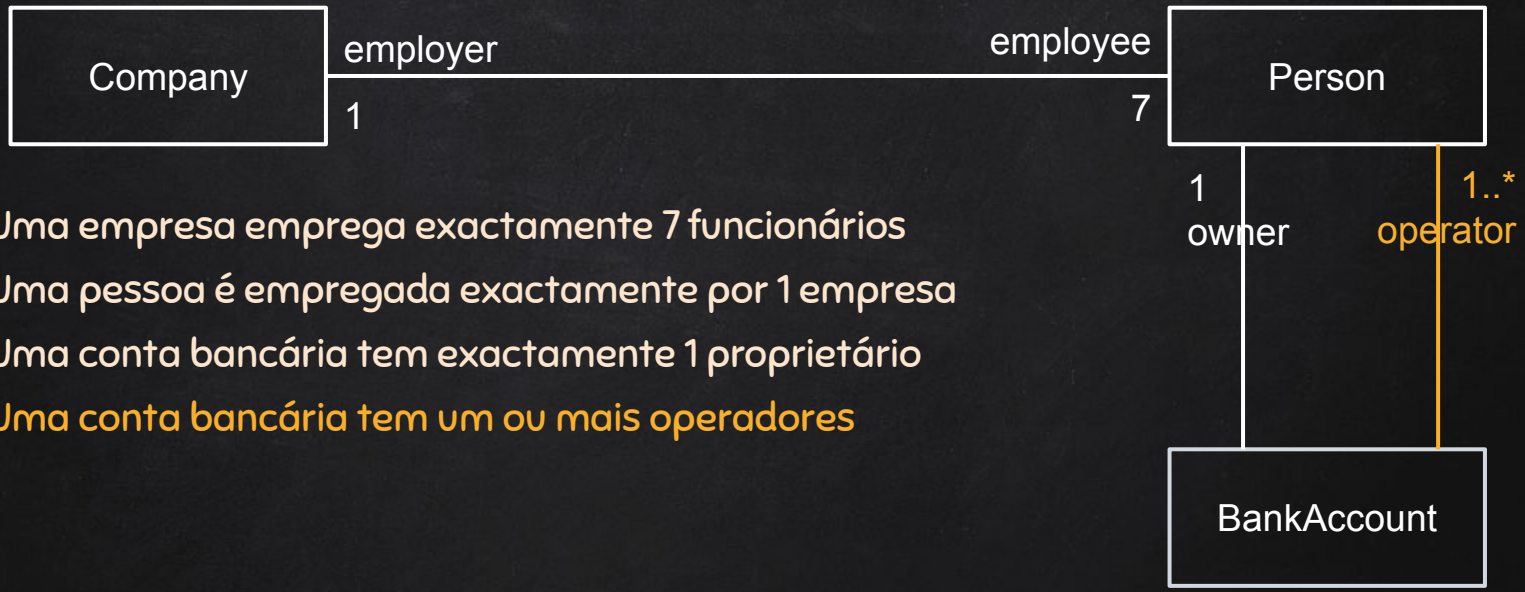


Uma empresa emprega exactamente 7 funcionários

Uma pessoa é empregada exactamente por 1 empresa

Uma conta bancária tem exactamente 1 proprietário

Certifique-se de que lê associações e multiplicidades com precisão



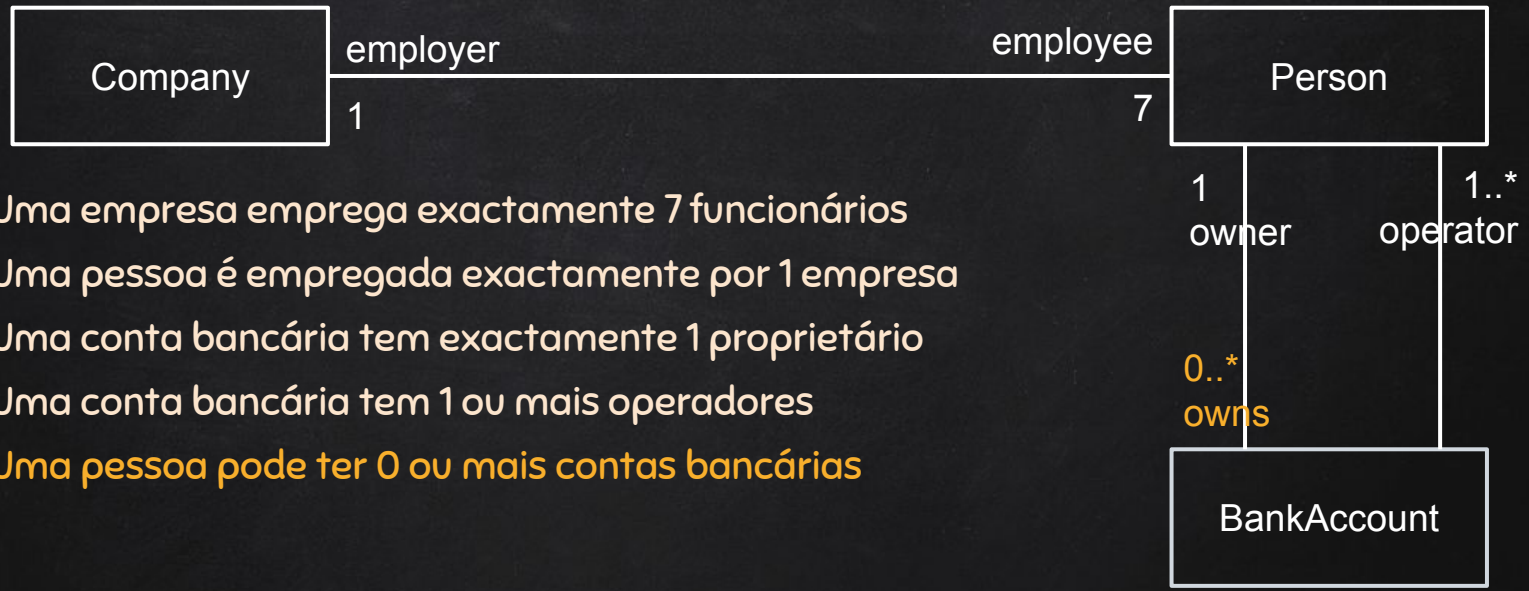
Uma empresa emprega exactamente 7 funcionários

Uma pessoa é empregada exactamente por 1 empresa

Uma conta bancária tem exactamente 1 proprietário

Uma conta bancária tem um ou mais operadores

Certifique-se de que lê associações e multiplicidades com precisão



Uma empresa emprega exactamente 7 funcionários

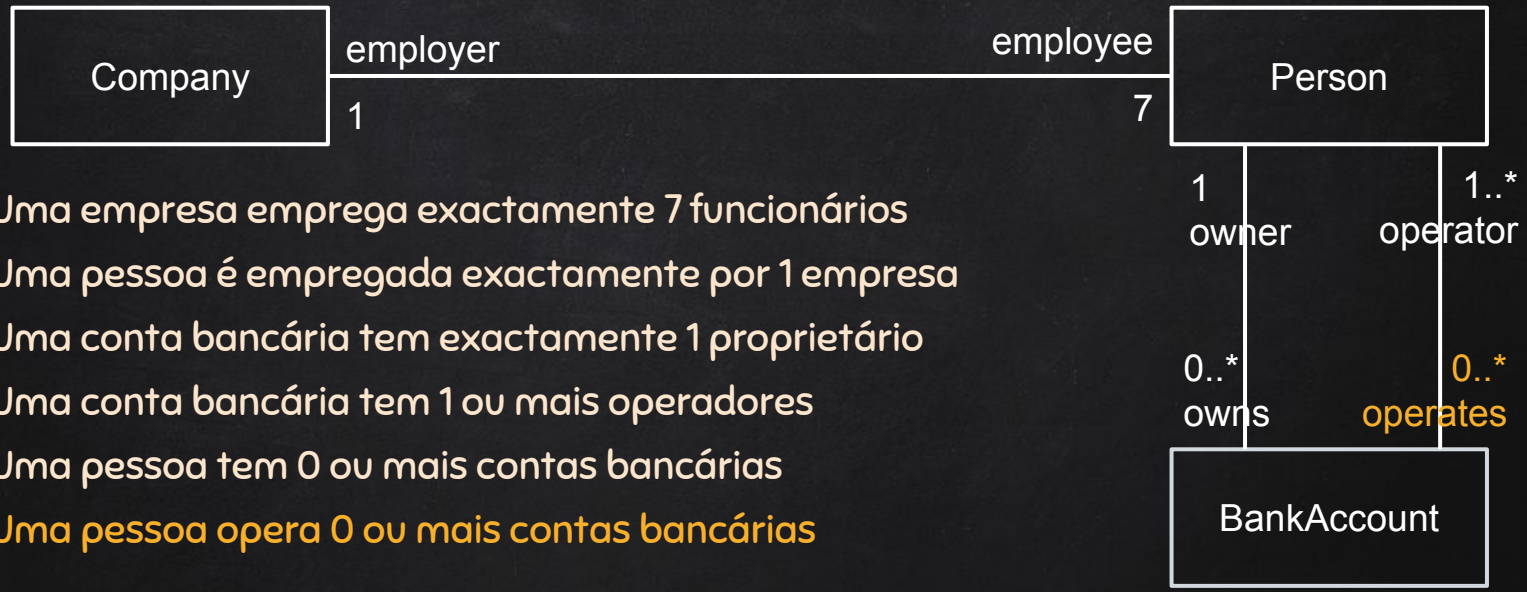
Uma pessoa é empregada exactamente por 1 empresa

Uma conta bancária tem exactamente 1 proprietário

Uma conta bancária tem 1 ou mais operadores

Uma pessoa pode ter 0 ou mais contas bancárias

Certifique-se de que lê associações e multiplicidades com precisão



Uma empresa emprega exactamente 7 funcionários

Uma pessoa é empregada exactamente por 1 empresa

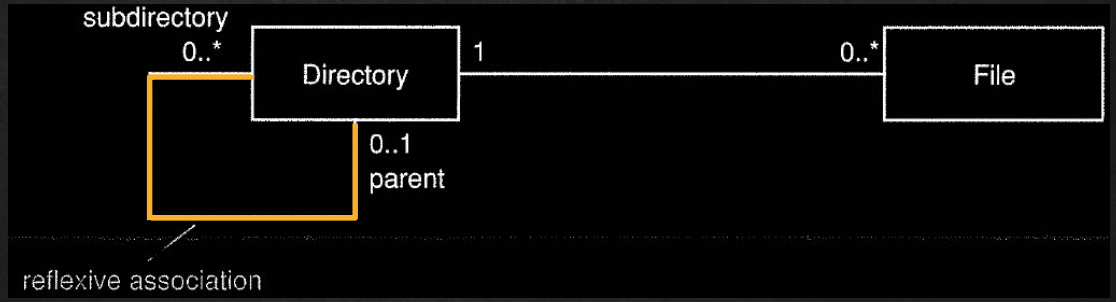
Uma conta bancária tem exactamente 1 proprietário

Uma conta bancária tem 1 ou mais operadores

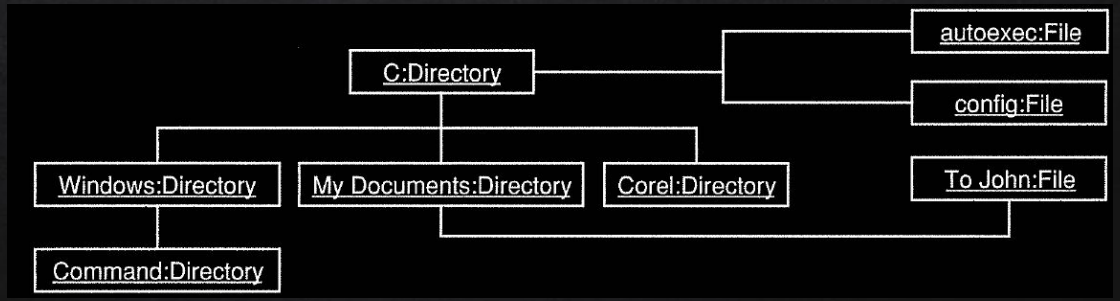
Uma pessoa tem 0 ou mais contas bancárias

Uma pessoa opera 0 ou mais contas bancárias

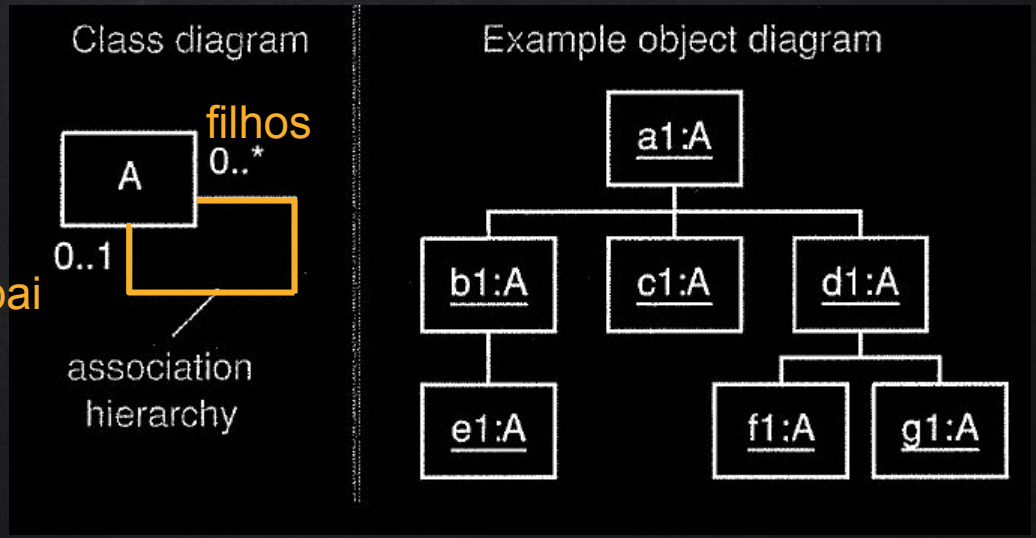
As associações reflexivas são associações à mesma classe



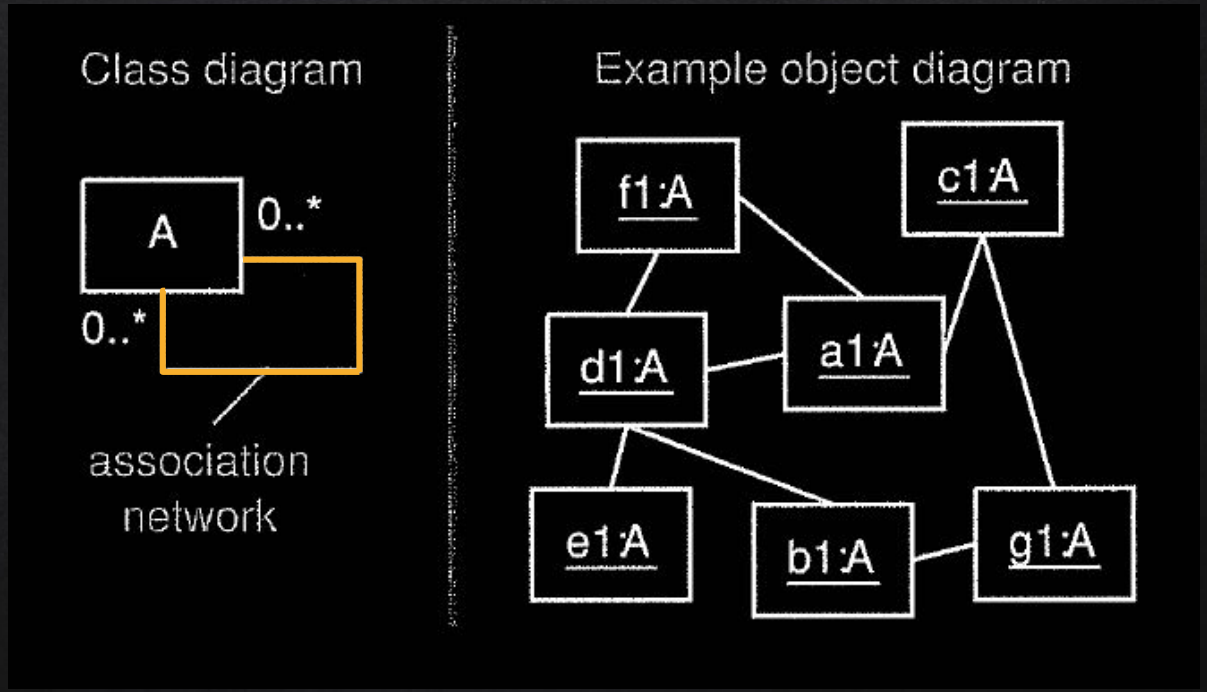
Este modelo pode ser instanciado da seguinte forma:



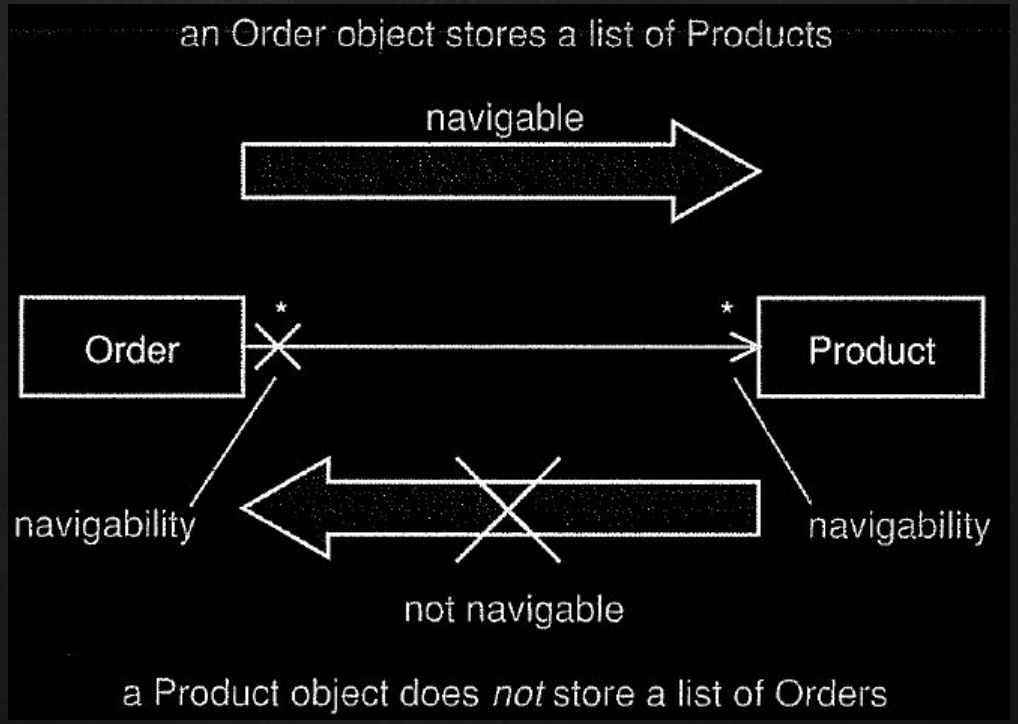
Numa hierarquia, um objecto pode ter 0 ou 1 objectos diretamente acima dele (pai).
O objecto pode ter 0 ou mais filhos.



Numa rede, cada nó pode ter de 0 ou mais objectos directamente ligados a ele



A navegabilidade indica que os objectos na classe de origem "conhecem" os objectos na classe de destino



O UML tem três idiomas alternativos para navegabilidade

- Navegabilidade estrita do UML 2 – todas as cruzes e setas são usadas explicitamente
  - Esta é a alternativa mais precisa

- Sem navegabilidade – não são utilizadas cruzes nem setas
  - Esta é a alternativa mais ambígua

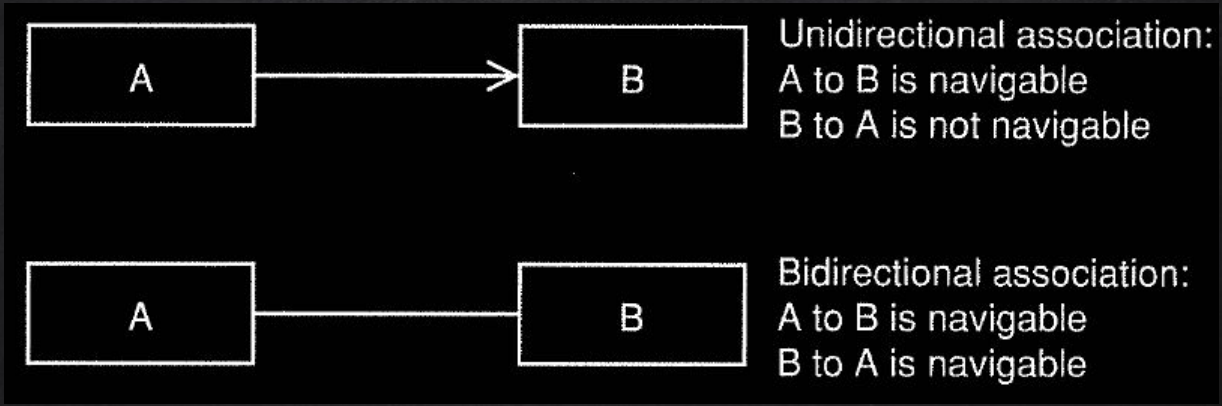
- Prática padrão – apenas as setas estão definidas
  - É uma expressão idiomática de compromisso e a mais usada na prática (na indústria e nesta disciplina)

Comparação entre expressões alternativas

Adoptado nesta disciplina

UML 2 navigability idioms			
UML 2 syntax	Idiom 1: Strict UML 2 navigability	Idiom 2: No navigability	Idiom3: Standard practice
	A to B is navigable B to A is navigable		
	A to B is navigable B to A is not navigable		
	A to B is navigable B to A is undefined		A to B is navigable B to A is not navigable
	A to B is undefined B to A is undefined	A to B is undefined B to A is undefined	A to B is navigable B to A is navigable
	A to B is not navigable B to A is not navigable		

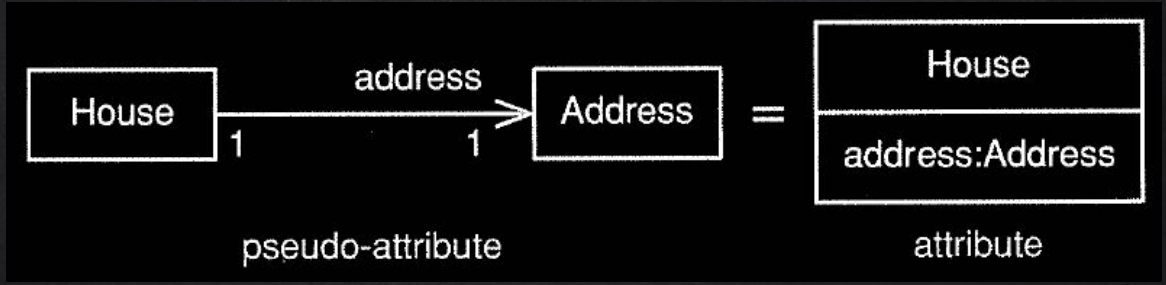
Adoptada nesta disciplina



Advertências:

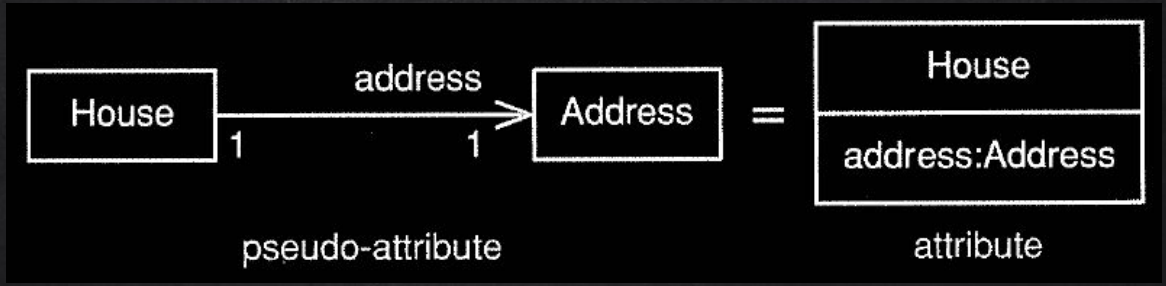
- É impossível saber se a navegabilidade está totalmente definida com esta expressão idiomática
- Altera a semântica de uma relação unidirecional de *Indefinido->Navegável* para *Não Navegável->Navegável*
- É impossível mostrar associações não navegáveis em ambos os sentidos (mas estas não são utilizadas na prática)

Associações de classes e atributos de classes



- O objecto da classe de origem tem uma referência para um objecto da classe de destino
- Se a associação tiver uma função na classe de destino, essa função corresponde a um pseudo-atributo na classe de origem, que pode ser transformado num atributo
- Neste exemplo, a classe *House* tem um pseudo-atributo chamado *address*, do tipo *Address*

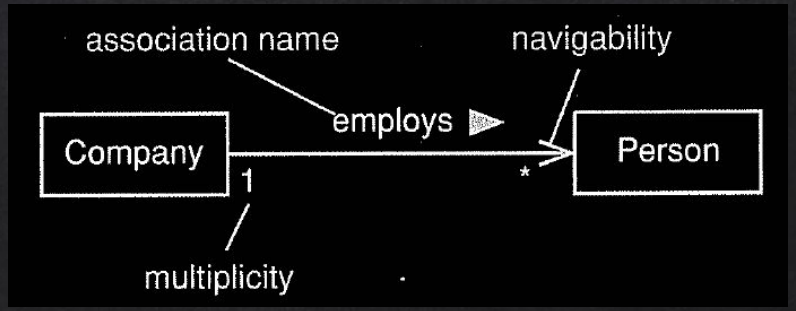
Esta associação pode ser transformada em código!




```
public class House {
   Address address;
}

public class Address {
}
```

Com outras cardinalidades, poderá ser necessária uma coleção de algum tipo (e.g., Collection ou Array)



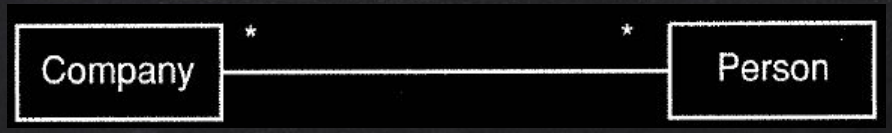

```
public class Company {
   Collection<Person> person;
}

public class Person {
}
```

```
public class Company {
   Person[] person;
}

public class Person {
}
```

Por que precisamos de classes de associação?

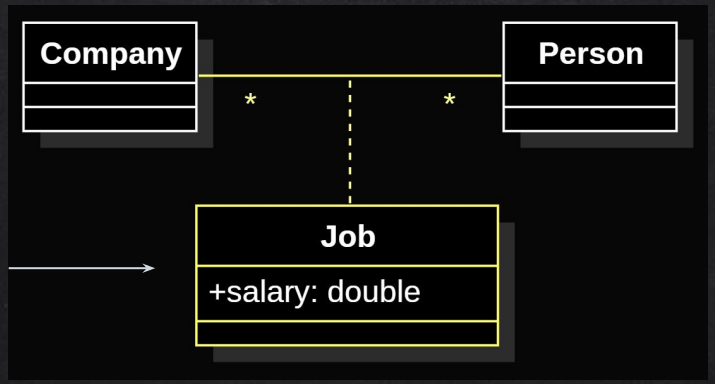


Em associações muitos-para-muitos, o que pode ser feito se for necessário atribuir um valor (e.g., uma salário) à associação?

- *Person* pode trabalhar para mais de uma *Company*
- Se for esse o caso, não podemos armazenar informações sobre o salário em nenhuma das classes
  - Se armazenarmos o salário na empresa, qual salário corresponde a qual pessoa?
  - Se armazenarmos o salário na pessoa e a pessoa trabalhar em mais de uma empresa, qual salário corresponde a essa pessoa em particular

A propriedade salary é melhor representada dentro de uma classe de associação

Classe de associação

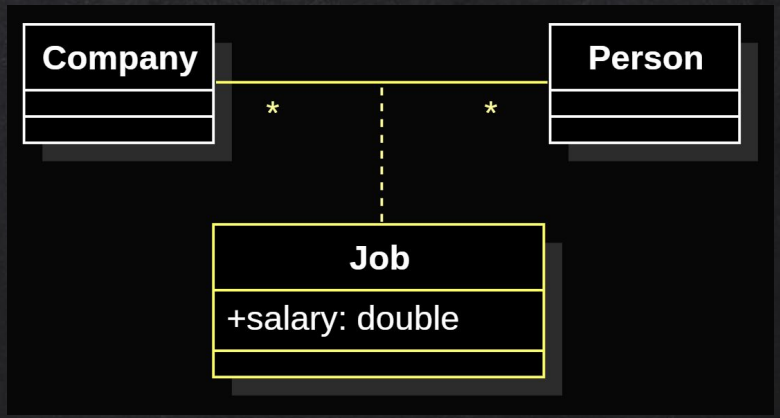


A classe de associação consiste na classe, na associação e na linha tracejada.

- Só pode haver uma ligação entre duas instâncias (uma da *Company* e outra da *Person*)
- O atributo *salary* é armazenado numa classe de associação chamada *Job*
- A classe de associação tem, na prática, uma ligação com uma *Company* e outra com uma *Person*

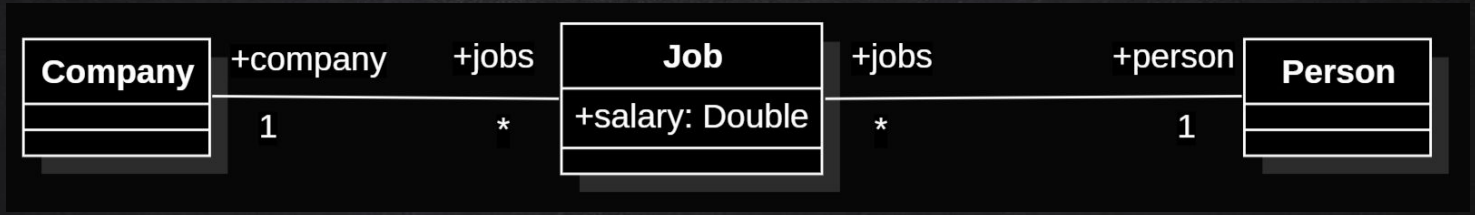
E se precisar de representar pessoas que trabalham em empresas diferentes ao mesmo tempo?

Neste exemplo...



- A Microsoft pode oferecer vários empregos a pessoas diferentes.
- O John só pode ter um emprego na Microsoft.
- O John pode ter diferentes empregos em diferentes empresas.

A classe de associação pode ser reificada (ou seja, Job pode ser transformada numa classe real)



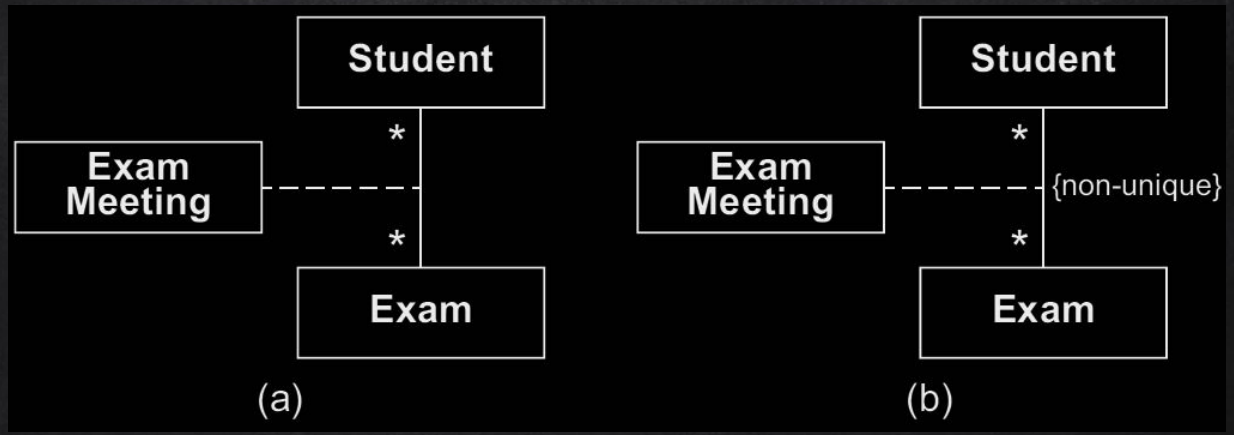

```
public class Company {
   Collection<Job> jobs;
}
```

```
public class Person {
   Collection<Job> jobs;
}
```

```
public class Job {
   Company company;
   Person person;
   double salary;
}
```

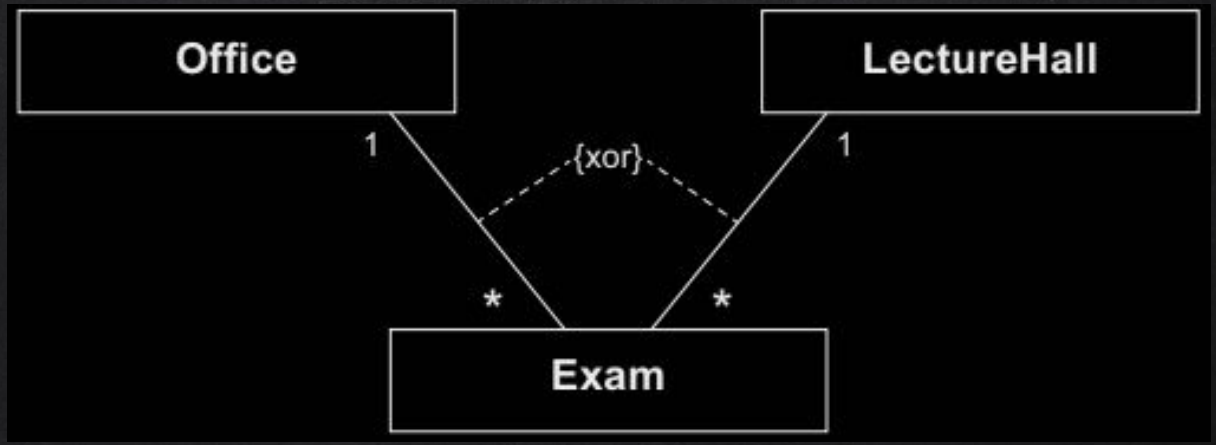
Atenção: isto tem quase a mesma semântica que a classe de associação. No entanto, desta forma, pode existir mais do que uma ligação entre uma determinada empresa e uma determinada pessoa num determinado momento! Precisamos de adicionar uma regra OCL (Object Constraint Language) para ter a mesma semântica ao nível do diagrama, e o código Java deve agir em conformidade.

Outro exemplo



a) Um aluno só pode ter uma reunião de exame para discutir o resultado do seu exame escrito uma vez
b) Um aluno pode ter mais do que um exame por reunião (graças ao uso da tag "non-unique")

Associações com restrições XOR



Um objecto da classe `Exam` deve ser associado a um objecto da classe `Office` ou a um objecto da classe `LectureHall`, mas não a ambos.

Why did the developer go broke?

Because he used up all his cache.

Referências

UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design

2ª edição, Jim Arlow and Ila Neustadt, Pearson Education

Capítulos 7, 8, 9 e 10

